

Conception de l'Architecture du Moteur de Jeu Zelda-like

1. Introduction

Ce document détaille l'architecture proposée pour transformer le prototype *Zelda-like* en un moteur de jeu robuste, capable de gérer des mondes complexes, des ennemis variés et une progression de type *A Link to the Past*. L'objectif est de passer d'un simple carré vert à un jeu complet, en se basant sur le code C/SDL2 existant et en intégrant les fonctionnalités requises par le cahier des charges.

2. Objectifs Clés

L'architecture sera conçue pour supporter les fonctionnalités suivantes :

- **Système de Monde (Multi-monde) :** Gestion d'un "Light World" et d'un "Dark World" avec un système de transition par miroir.
- **Bestiaire Avancé :** Système d'IA pour les ennemis (états : patrouille, poursuite, attaque).
- **Moteur d'Événements :** Un système pour déclencher des boss, ouvrir des portes avec des clés et gérer des énigmes.
- **HUD Complet :** Affichage des cœurs, rubis, jauge magique, bombes et carte/menu.
- **Progression :** Système de quêtes principales (trois pendentifs), sous-quêtes et secrets (flûte, grappin, morceaux de cœur).

3. Structure Modulaire du Projet

Le projet sera organisé en modules distincts pour améliorer la maintenabilité, la lisibilité et la scalabilité. Voici la structure des répertoires et les modules principaux :

Plain Text

```
zelda-like-mvp/  
├── assets/  
│   ├── sprites/      # Sprites des personnages, ennemis, objets, tiles  
│   ├── maps/         # Fichiers de données des cartes (Light World, Dark World)  
│   ├── sounds/       # Effets sonores et musiques  
│   └── include/      # Fichiers d'en-tête (.h)  
│       ├── game_config.h # Constantes globales (taille écran, tuiles, etc.)  
│       ├── types.h      # Définitions de types communs (Direction, GameState, Hit)  
│       ├── player.h     # Définitions et fonctions du joueur  
│       └── enemy.h      # Définitions et fonctions des ennemis
```

```

|   |— particle.h      # Définitions et fonctions du système de particules
|   |— game_map.h     # Définitions et fonctions de la carte du jeu
|   |— game.h         # Structure principale du jeu et fonctions de gestion
|   |— world.h        # Gestion des mondes (Light/Dark) et transitions
|   |— entity.h       # Base pour toutes les entités (joueur, ennemis, objets)
|   |— event_engine.h # Moteur d'événements
|   |— hud.h          # Affichage du HUD
|   |— item.h         # Gestion des objets et de l'inventaire
|   |— dungeon.h      # Gestion des donjons et de leurs spécificités
|— src/               # Fichiers source (.c)
|   |— player.c
|   |— enemy.c
|   |— particle.c
|   |— game_map.c
|   |— game.c
|   |— world.c
|   |— entity.c
|   |— event_engine.c
|   |— hud.c
|   |— item.c
|   |— dungeon.c
|— main.c             # Point d'entrée principal du programme
|— Makefile           # Fichier de compilation (ou CMakeLists.txt)
|— README.md

```

4. Composants Clés de l'Architecture

4.1. Système de Monde (World System)

- **world.h / world.c** : Gérera le chargement et le déchargement des cartes pour le Light World et le Dark World. Il maintiendra l'état actuel du monde (Light ou Dark) et la position du joueur dans ce monde.
- **Transition par Miroir** : Une fonction spécifique `world_transition(Player* player)` sera implémentée pour gérer le changement entre les deux mondes. Cela impliquera le chargement d'une nouvelle carte et la mise à jour de la position du joueur en fonction du point de transition.
- **Structure de Données de Carte** : Chaque carte sera représentée par une structure `GameMap` étendue pour inclure des informations spécifiques au monde (par exemple, des points de spawn d'ennemis, des zones de transition).

4.2. Bestiaire Avancé (Advanced Bestiary)

- **enemy.h / enemy.c** : La structure `Enemy` sera étendue pour inclure des états d'IA (patrouille, poursuite, attaque, blessé, mort). Chaque type d'ennemi (Octorok, Moblin, Knight, Hinox, Tektite, Armos, Lynel, Goriya, Wizzrobe, Trinex) aura ses propres comportements définis dans des fonctions d'IA spécifiques.
- **Gestion des États** : Une machine à états simple sera implémentée pour chaque ennemi, permettant des transitions fluides entre les comportements. Par exemple, un ennemi en patrouille passera en mode poursuite si le joueur entre dans son rayon de détection.
- **Attaques et Collisions** : Le système de collision existant sera amélioré pour gérer les interactions entre le joueur, les ennemis et leurs attaques respectives.

4.3. Moteur d'Événements (Event Engine)

- **event_engine.h / event_engine.c** : Ce module sera responsable de la gestion des événements scriptés dans le jeu. Cela inclura :
 - **Déclencheurs (Triggers)** : Zones spécifiques sur la carte qui, lorsqu'elles sont traversées par le joueur, déclenchent un événement (par exemple, apparition d'un boss, ouverture d'une porte).
 - **Conditions** : Vérification de prérequis pour un événement (par exemple, avoir une clé spécifique pour ouvrir une porte, avoir battu un certain nombre d'ennemis).
 - **Actions** : Les conséquences d'un événement (par exemple, changer l'état du jeu, modifier la carte, faire apparaître un objet).
- **Boss** : Les combats de boss seront gérés comme des événements spéciaux, avec des phases et des patterns d'attaque définis.
- **Énigmes** : Des énigmes simples (par exemple, pousser des blocs, activer des interrupteurs) seront intégrées via le moteur d'événements.

4.4. HUD (Head-Up Display)

- **hud.h / hud.c** : Ce module gèrera l'affichage de toutes les informations du joueur à l'écran :
 - **Cœurs** : Représentation visuelle de la santé du joueur (max 20).
 - **Rubis** : Compteur de monnaie (max 999).
 - **Jauge Magique** : Pour les capacités spéciales.
 - **Bombes** : Compteur de bombes.
 - **Équipement** : Affichage de l'objet équipé.
 - **Carte/Menu** : Accès à une carte du monde et un menu d'inventaire.

4.5. Système d'Objets et d'Inventaire (Item and Inventory System)

- `item.h` / `item.c` : Gèrera la collecte, l'utilisation et l'affichage des objets. Cela inclura les clés, les bombes, le grappin, la flûte, etc.
- **Inventaire** : Une structure de données pour stocker les objets collectés par le joueur.

4.6. Donjons (Dungeons)

- `dungeon.h` / `dungeon.c` : Chaque donjon aura sa propre structure de carte, ses ennemis spécifiques, ses énigmes et son boss. Ce module gèrera le chargement et la logique spécifique à chaque donjon.

5. Gestion des Assets

Le répertoire `assets/` sera structuré pour contenir tous les éléments graphiques, sonores et de données du jeu. Les sprites seront chargés et gérés par un système de texture manager pour optimiser la mémoire et les performances.

6. Compilation et Déploiement

La compilation sera gérée via un `Makefile` (ou un script `build.sh` comme alternative) utilisant Emscripten pour cibler le web (HTML5/WebAssembly). Cela permettra de déployer le jeu facilement sur des navigateurs web.

7. Prochaines Étapes

1. **Implémentation du Moteur de Base** : Développer les fonctionnalités fondamentales (tiles, caméra, collisions, HUD, système de monde).
2. **Implémentation du Joueur** : Mouvement, attaques, gestion des cœurs et des objets.
3. **Implémentation du Bestiaire** : Intégration des différents types d'ennemis avec leur IA.
4. **Développement du Moteur d'Événements** : Création des déclencheurs, conditions et actions.
5. **Intégration des Mondes et Donjons** : Chargement des cartes, transitions et logiques spécifiques.

Ce plan d'architecture fournit une feuille de route claire pour le développement du jeu, en s'appuyant sur une approche modulaire et évolutive.